

UNIVERSIDADE FEDERAL DE SANTA CATARINA
CENTRO DE CIÊNCIAS, TECNOLOGIAS E SAÚDE
BACHARELADO EM
TECNOLOGIAS DA INFORMAÇÃO E COMUNICAÇÃO

RELATÓRIO DE MANUTENÇÃO E EVOLUÇÃO GERENCIADOR DE TAREFAS KANBAN

Ana Beatriz Paschoali Boselo
Nathalia Paes Matias

Araranguá
2026

1. IDENTIFICAÇÃO E CARACTERIZAÇÃO DO PROJETO

Projeto: Gerenciador de Tarefas Kanban.

Projeto original:

https://github.com/1laryssaferreira/Gerenciador-de-tarefas-Kanban/blob/main/src/routes.py

O sistema é uma aplicação web simples de gerenciamento de tarefas no estilo Kanban. As tarefas são organizadas em três estados: To Do, In Progress e Done. O projeto utiliza Python e Flask, com interface HTML/CSS incorporada ao código e testes automatizados com Pytest.

O projeto foi escolhido por possuir escopo pequeno, execução local simples e código suficiente para análise de manutenibilidade. A estrutura também permitiu observar problemas de separação de responsabilidades, duplicação de lógica e concentração de regras em rotas.

2. BASELINE

Antes das modificações, a versão original foi mantida na branch main e utilizada como baseline do trabalho. O commit de referência é 6040c8f.

Informação	**Resultado**
Commit da baseline	6040c8f
Branch da baseline	main
Build / configuração	Sucesso (projeto Python/Flask, sem etapa de compilação específica)
Quantidade de testes	10
Testes executados com sucesso	10 de 10
Execução da aplicação	Sucesso

3. ANÁLISE DE MANUTENIBILIDADE

Foram identificados três problemas principais de manutenibilidade no código ampliado antes das refatorações. Os problemas foram escolhidos por aumentarem o esforço de compreensão e alteração futura do sistema.

1. Mistura de responsabilidades no routes.py

Problema identificado: o arquivo routes.py concentrava as rotas Flask, a interface HTML/CSS e também a lógica de persistência em JSON.

Localização: src/routes.py.

Evidência: o mesmo módulo possuía ARQUIVO_DADOS e as funções carregar_tarefas() e salvar_tarefas(), ao mesmo tempo em que declarava os endpoints e os templates da aplicação.
Consequência: alterações na forma de armazenamento exigiam modificar o mesmo módulo responsável pelas rotas, aumentando o acoplamento e dificultando a manutenção.
Possível tratamento: extrair a persistência para um módulo próprio, aplicando separação de responsabilidades.

2. Duplicação na alteração de status

Problema identificado: os métodos iniciar() e concluir() repetiam a mesma sequência de operações para alterar o status e registrar a mudança no histórico.
Localização: classe Task, em src/models.py.
Evidência: ambos os métodos armazenavam o status anterior, atribuíam o novo status e adicionavam uma mensagem semelhante ao histórico.
Consequência: uma mudança futura na regra de transição ou no formato do histórico precisaria ser repetida em mais de um local, aumentando o risco de inconsistências.
Possível tratamento: extrair a lógica repetida para um método auxiliar único.

3. Rota de edição com regras de negócio

Problema identificado: a rota editar() realizava comparações de todos os campos, registrava alterações no histórico e atualizava diretamente os atributos da tarefa.
Localização: função editar() em src/routes.py.
Evidência: a rota continha várias verificações condicionais para título, descrição, prioridade, categoria e data limite, além das atribuições dos novos valores.
Consequência: a rota ficou extensa e passou a concentrar regras que pertencem à entidade Task, reduzindo a legibilidade e dificultando o reaproveitamento e o teste isolado dessa lógica.
Possível tratamento: mover a atualização e o registro de histórico para um método da classe Task.

4. REFATORAÇÕES REALIZADAS

As três refatorações foram realizadas separadamente e registradas em commits distintos para permitir rastreabilidade. Após as alterações, os testes existentes foram executados novamente para verificar a preservação do comportamento.

1. Extração da persistência para storage.py

Técnica/estratégia: extração de módulo e separação de responsabilidades.

Alteração: as funções carregar_tarefas() e salvar_tarefas(), além da definição do caminho do arquivo JSON, foram removidas de routes.py e transferidas para o novo módulo src/storage.py.

Justificativa: a persistência passou a ter um local próprio, reduzindo as responsabilidades de routes.py e facilitando mudanças futuras na forma de armazenamento.

Commit: d5ff51d — refactor: separa persistência json das rotas.

2. Centralização da alteração de status

Técnica/estratégia: extração de método.

Alteração: foi criado o método privado _alterar_status() na classe Task. Os métodos iniciar() e concluir() passaram apenas a informar o novo status desejado.

Justificativa: a lógica comum ficou centralizada, eliminando duplicação e reduzindo o risco de comportamentos diferentes entre transições de status.

Commit: af58934 — refactor: centraliza alteração de status.

3. Movimentação da lógica de edição para Task

Técnica/estratégia: movimentação de método / encapsulamento de comportamento.

Alteração: foi criado o método atualizar() na classe Task. A rota editar() passou a receber os valores do formulário, chamar tarefa.atualizar(...) e persistir o resultado.

Justificativa: as regras de atualização e de registro do histórico ficaram na entidade responsável pelos dados, deixando a rota menor e mais clara.

Commit: 956dc51 — refactor: move logica de edicao para Task.

Resumo das refatorações

Problema	Localização	Tratamento	Commit
Persistência misturada às rotas	routes.py	Extração de módulo	d5ff51d
Duplicação na mudança de status	models.py	Extração de método	af58934
Lógica extensa na edição	routes.py / models.py	Movimentação de método	956dc51

5. SOLICITAÇÃO DE MANUTENÇÃO PERFECTIVA

Descrição da necessidade: no projeto original, após a criação de uma tarefa não havia uma forma direta de corrigir ou atualizar seus dados. A necessidade identificada foi permitir que o usuário alterasse uma tarefa existente sem precisar excluí-la e cadastrá-la novamente.

Comportamento esperado: ao selecionar a opção Editar em uma tarefa, o sistema deve apresentar os dados atuais, permitir a alteração de título, descrição, prioridade, categoria e data limite e, após o salvamento, retornar ao quadro Kanban exibindo os novos valores.

Regras e restrições: a edição deve atuar sobre a tarefa selecionada; o status atual deve ser preservado; as alterações devem ser persistidas no arquivo JSON; e a edição não deve eliminar as demais informações da tarefa. Quando houver mudança nos dados, o histórico deve registrar a alteração correspondente.

Implementação: a evolução funcional foi registrada no commit 9b48ce4 — feat: adiciona funcionalidades ao kanban, juntamente com outras pequenas melhorias agregadas ao projeto. Posteriormente, a estrutura interna relacionada à edição foi melhorada pela refatoração registrada no commit 956dc51.

6. ANÁLISE DE IMPACTO

Como a sequência das etapas foi invertida conforme orientação em aula, a análise abaixo registra o impacto que seria esperado para a solicitação de edição e o compara com o impacto efetivamente observado durante a implementação e a refatoração. A distinção entre previsão e resultado foi mantida para atender ao objetivo de análise de impacto do trabalho.

Impacto previsto

A funcionalidade de edição foi considerada de baixo a médio impacto, pois utilizaria os dados já existentes na entidade Task e exigiria principalmente mudanças na interface e nas rotas da aplicação.

Elemento	Impacto previsto
src/routes.py	Adicionar rota de edição, receber dados do formulário e salvar as alterações.
Interface HTML	Adicionar botão Editar e formulário com os dados atuais da tarefa.
src/models.py	Verificar se a entidade Task precisaria de suporte adicional para atualização dos campos.
src/storage.py / JSON	Reutilizar a persistência existente para gravar os valores atualizados.
Testes	Executar a suíte existente e verificar manualmente o novo fluxo de edição.

Impacto observado

O impacto observado ficou próximo do previsto. O arquivo routes.py recebeu a rota /editar/<int:id> e a tela de edição. A persistência já existente foi reutilizada para salvar a tarefa atualizada. Durante a refatoração, models.py também passou a concentrar a lógica de atualização por meio do método Task.atualizar(), reduzindo a quantidade de regras dentro da rota.

O módulo storage.py não precisou de uma regra específica para edição: a função salvar_tarefas() continuou sendo utilizada normalmente. A estrutura do arquivo JSON também foi mantida, pois os campos editados já faziam parte dos dados persistidos.

7. VERIFICAÇÃO E TESTES

A baseline do projeto possuía 10 testes automatizados em Pytest. Durante o processo de refatoração, a suíte foi executada novamente e foi obtido o resultado de 10 testes aprovados, sem falhas, fornecendo evidência de preservação dos comportamentos cobertos pelos testes existentes.

Verificações funcionais realizadas:

- criação de tarefas com prioridade, categoria e data limite;
- edição dos dados de uma tarefa existente;
- movimentação entre To Do, In Progress e Done;
- registro das alterações no histórico;
- salvamento e carregamento das tarefas por meio do arquivo JSON.

Comparação com a baseline: a quantidade de testes automatizados existentes permaneceu em 10 e a execução registrada continuou apresentando 10 aprovações. Não foram identificadas regressões nos comportamentos cobertos pela suíte disponível

8. CONCLUSÃO

O trabalho permitiu compreender na prática que a manutenção de um software desenvolvido por terceiros exige primeiro conhecer sua estrutura e preservar o comportamento existente durante as alterações. A utilização de uma aplicação pequena facilitou a compreensão do fluxo, mas a inclusão de novas funcionalidades mostrou

rapidamente como um arquivo pode acumular responsabilidades e como duplicações surgem durante a evolução do código.

Entre as principais dificuldades estavam a preparação do ambiente em diferentes máquinas, a configuração inicial do Git, a organização das modificações em uma branch de manutenção e a correção de problemas de integração durante as refatorações. Erros de indentação e a ausência temporária de um método utilizado pela persistência também demonstraram a importância de executar a aplicação e os testes após mudanças estruturais.

As refatorações realizadas reforçaram três aprendizados principais: separar responsabilidades facilita futuras modificações; centralizar lógica repetida reduz o risco de inconsistências; e manter regras relacionadas à tarefa dentro da própria classe melhora a legibilidade e o reaproveitamento do código. O histórico de commits também foi importante para rastrear cada intervenção e distinguir a implementação funcional das melhorias estruturais.

Ao final, a branch main permaneceu como referência da versão original, enquanto a branch manutencao passou a concentrar as evoluções e refatorações realizadas. O projeto continuou executável e os testes existentes permaneceram aprovados na execução registrada, atendendo ao objetivo de evoluir o software mantendo controle sobre seus impactos.